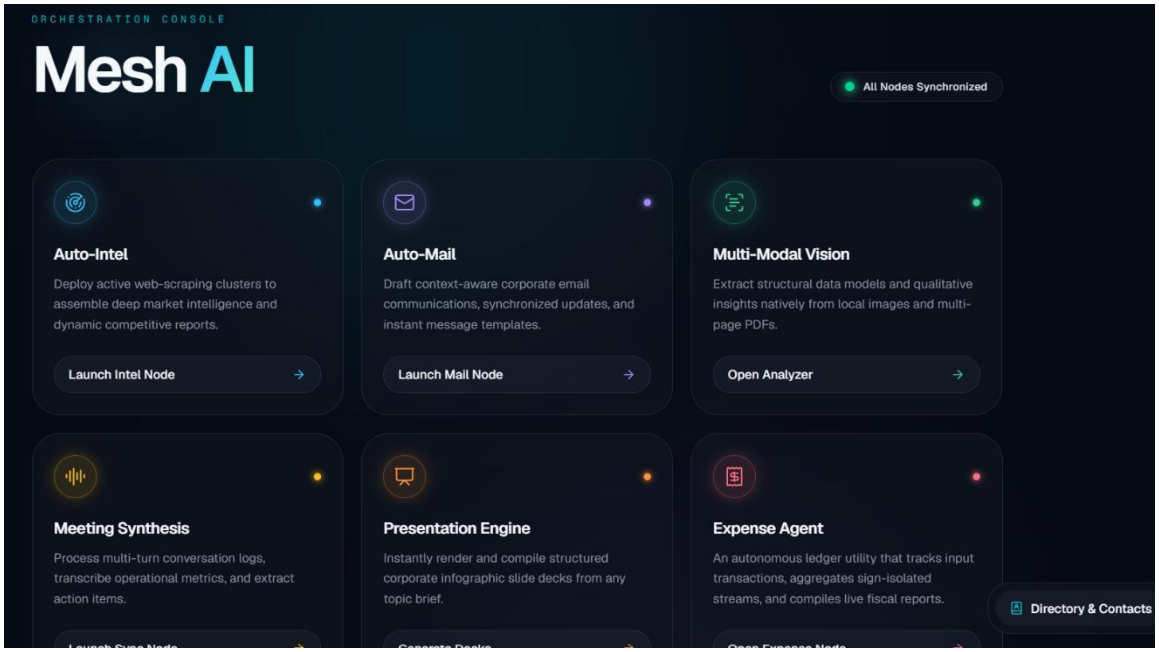
Mesh AI

Selected Track: Agents for Business

5-Minute Demo Video: https://youtu.be/KsM893WI0Oc

Public Project Link: https://github.com/MedhaanshA/MeshAIFinal



---

1. Problem Statement

Modern business teams juggle a fragmented stack of tools to stay informed and productive: one app for competitor research, another for meeting notes, another for expense tracking, another for drafting emails, and yet another for building presentations. Each of these tasks is repetitive, time-sensitive, and prone to human error when done manually under deadline pressure.

Mesh AI was built to collapse this fragmented workflow into a single, autonomous, multi-agent system. Rather than being a single chatbot bolted onto a UI, Mesh AI is architected as a **hub-and-spoke network of specialized agents**, each an expert at one job, coordinated by a central orchestrator. The result is a desktop-class business assistant that can research competitors, read documents and images, transcribe and summarize meetings, generate slide decks, draft emails, track expenses, and remember context, all through one interface, and all designed to keep working even when individual APIs fail or rate-limit.

---

2. System Architecture & Overall Working Flow

Mesh AI operates on a **hub-and-spoke multi-agent architecture**, using FastAPI as the central gateway and the Model Context Protocol (MCP) to distribute complex tasks to isolated, specialized servers. The system is built around three design priorities: resilience, autonomy, and native multimodal handling.

Phase 1: Ingestion & the API Gateway (`api_gateway.py`)

- **Client Request:** The Next.js frontend dashboard sends REST calls or multipart form data (files, audio, images) to the FastAPI backend.
- **Endpoint Routing:** The Gateway identifies intent (e.g. `/api/intel`, `/api/mail`, `/api/vision`, `/api/expense`) and routes to the correct handler.
- **Context Retrieval:** For conversational tasks, the system reads `chat_history.json` to restore prior context, giving the agent stateful memory across sessions.

Phase 2: Agentic Orchestration (`main_agent.py`)

The Main Orchestrator Agent, powered by Gemini, is the "brain" of the system:

- **Intent Analysis:** Determines which sub-agents or tools a request needs.
- **Constraint Formatting:** Injects strict system instructions, such as forcing JSON-only outputs, before passing work down the pipeline.

Phase 3: Execution Routing, Local Skills vs. MCP Servers

The Orchestrator splits work across two paths:

- **Path A, Native Local Skills (`agent_skills.py`):** Used for file processing, data visualization, and presentation generation, for example, parsing an uploaded PDF, generating a `.pptx` deck, or running Gemini Vision analysis on an image. Outputs are written directly to the frontend's `public` directory and their paths returned to the UI.
- **Path B, External MCP Servers (`mcp_search_server.py`, `mcp_mail_server.py`):** Used for isolated, specialized jobs like live web scraping or staging emails. The Orchestrator connects over Server-Sent Events (SSE) via `ClientSession` to independent FastMCP servers (ports 8001, 8002), which execute the tool and stream the result back.

Phase 4: The Resilience Layer

This is the backbone of the system's "vibe coding" robustness, designed to survive rate limits and environment conflicts without ever crashing:

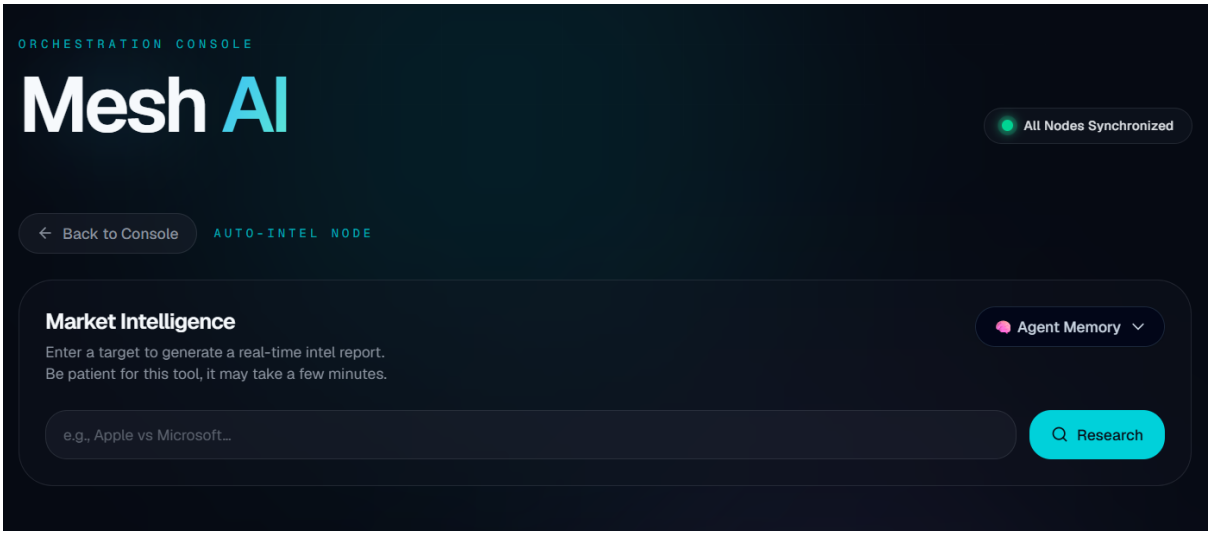
- **Automated Exponential Backoff:** Every Gemini and MCP call is wrapped in a `@retry_on_429` decorator, doubling wait time on `RESOURCE_EXHAUSTED` errors.
- **3-Layer Search Fallback:** DuckDuckGo, then Wikipedia, then offline compiled mock data, so the research pipeline never fully breaks.
- **Binary Conflict Fallback:** If `matplotlib` fails to load on the host, charting silently degrades to a pure Pillow (PIL) renderer.

Phase 5: Aggregation & Delivery

- **Payload Assembly:** The Orchestrator gathers raw text, scraped data, or file paths from the tools it called.
- **Final Polish:** Synthesizes everything into the requested output format: an intelligence report, an action-item array, a formatted email draft, etc.
- **Response:** The Gateway returns the final payload to the client and persists new state to `chat_history.json` and `records.json`.

---

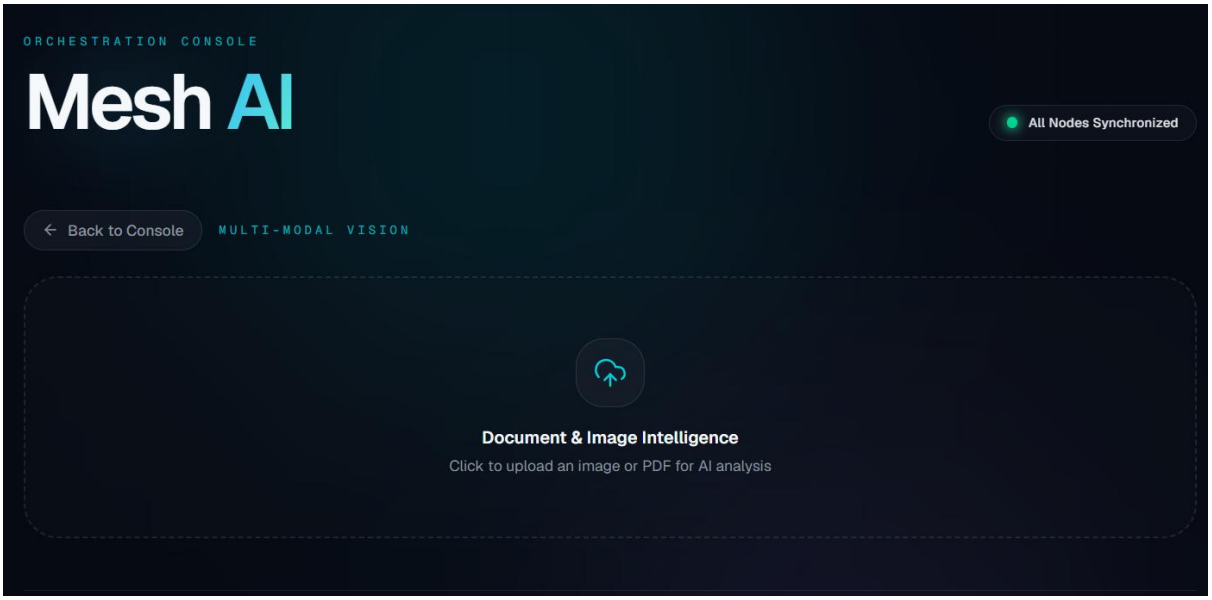
3. The Agent & Tool Suite



A two-agent pipeline backed by MCP servers:

- **Research Agent (`search_and_scrape_web`):** A resilient web-scraping agent with a 3-layer fallback: DuckDuckGo live search, Wikipedia API, and an offline structured-mock compiler, so intelligence gathering survives rate limits.
- **Compliance Writer Agent:** Takes the Research Agent's raw output and restructures it into a strict, UI-ready JSON payload with executive summaries and actionable opportunities.

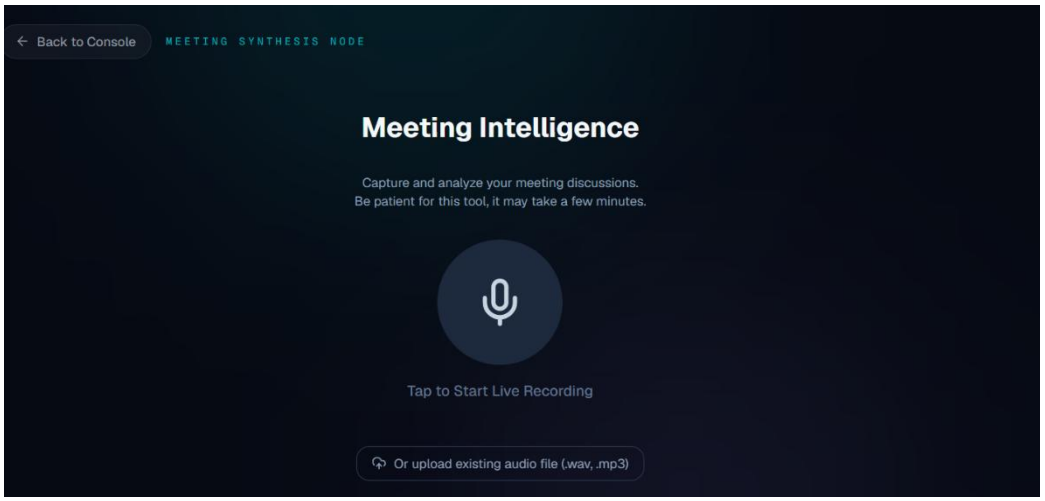
3.2 Multimodal Analysis (Vision & Document Processing)



Built on the Gemini APIs:

- **Vision AI Tool (`process_vision_image`):** Zero-shot image analysis via Gemini 2.5 Flash, returning a 2-sentence summary, 3 key visible elements, and one actionable insight.
- **Document AI Tool (`process_document_file`):** Parses PDFs, extracting dense text into structured executive takeaways.
- Both tools enforce a **cloud cleanup protocol**, deleting uploaded files immediately after processing.

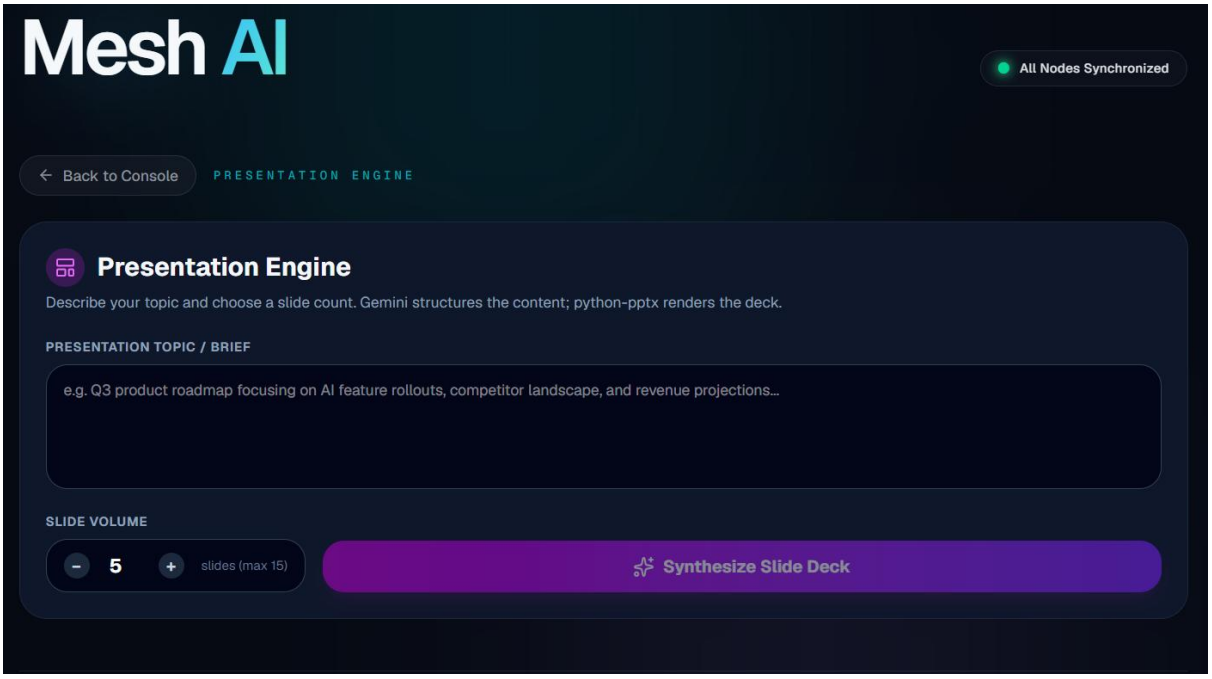
3.3 2-Step Audio Pipeline (Meeting Summarizer)



Handles chunked/streamed `.webm` meeting recordings through two stages:

- **Transcription Engine:** Uses `gemini-2.5-flash-lite` for fast, accurate speech-to-text.
- **Executive Insight Generator:** A second agent pass reads the transcript and produces an executive summary plus a prioritized JSON array of Action Items (Task, Priority, Due Date, Owner).

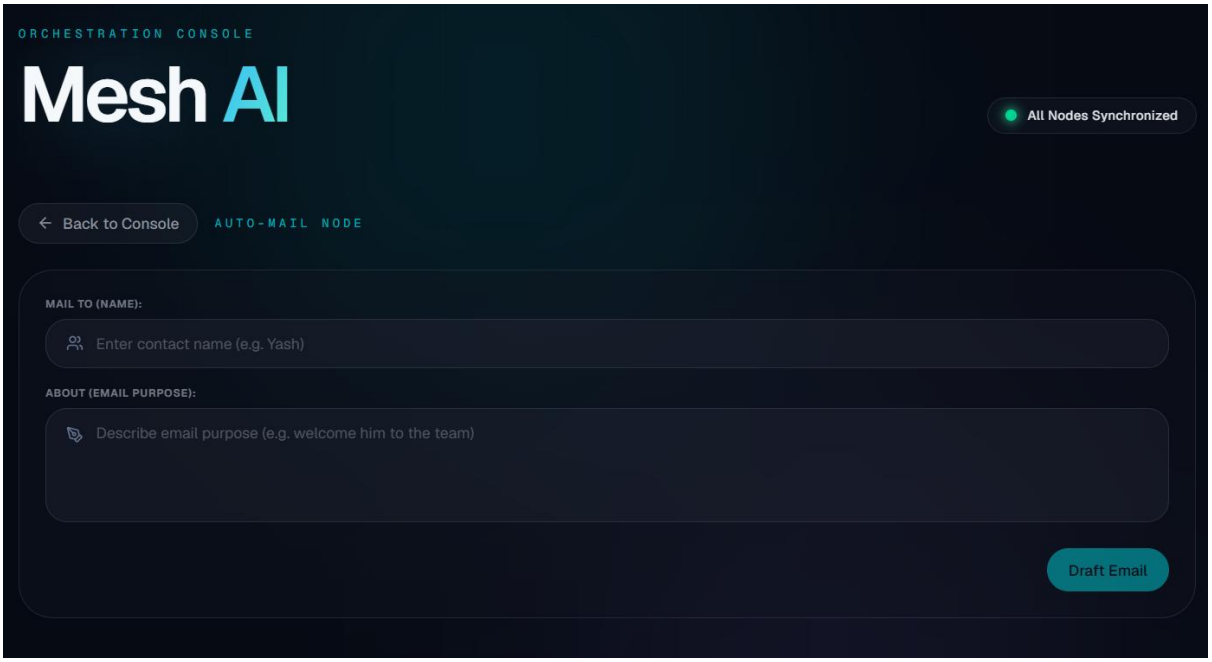
3.4 Generative Presentation Tool (`generate_slide_deck`)



Converts raw prompts into a fully rendered, styled `.pptx` deck using `python-pptx`:

- **Agentic Layout Engine:** Forces the AI, via a strict JSON schema, to pick the optimal layout per slide (`TIMELINE`, `3_COLUMN_GRID`, `KPI_CARDS`, `TEXT_DEFAULT`).
- **Automated Styling:** Applies a corporate "dark slate" aesthetic, headers, dynamic slide numbering, and custom shapes, with no PowerPoint installation required on the host.

3.5 MCP-Powered Communication (Auto-Mail Engine)



- **Mail Drafter (`draft_and_stage_email`):** Hosted on an independent FastMCP server (port 8002), drafts professional emails with dynamically generated, context-aware subject lines.
- **Security Guardrails (`execute_input_guardrail`, `_guardrail_check`):** Silently evaluate both user input and LLM output to block inappropriate, fraudulent, or illicit content before any email is drafted or staged.

3.6 Expense Tracking & Dynamic Charting

- **Ledger Tool:** Records credits/debits, calculates aggregate balances, and maintains transaction history (`records.json`).
- **Data Visualization Agent (`generate_expense_chart`):** Renders a styled, dark-themed transaction bar chart, defaulting to Matplotlib and falling back to Pillow (PIL) if system binaries conflict.

3.7 Stateful Memory & Context Management

- **Memory History Engine:** Endpoints (`/api/intel/history`) to save, retrieve, and wipe conversational memory (`chat_history.json`), giving the agent persistent context across sessions and workflows.

3.8 Core System Utilities

- **Micro-Retry & Backoff Wrappers (`@retry_on_429`):** Custom decorators handling HTTP 429 errors via automated exponential backoff.
- **Mock Entity Database:** An in-memory Contacts/Teams map (e.g. Executive Leadership, Core Engineering) for fast lookup and routing during agent tasks, such as auto-populating recipients for the Auto-Mail Engine.

---

4. How the Agents Trigger Their Tools

Every tool call in Mesh AI follows the same disciplined trigger pattern:

1. The Orchestrator receives an intent-classified request from the Gateway.
2. It formats a constrained prompt (often forcing strict JSON output) and decides: local skill or MCP server?
3. Local skills execute synchronously in-process (vision, documents, charts, decks).
4. MCP-routed tools are invoked over SSE to an isolated FastMCP server, which runs the tool, wraps the result, and streams it back.
5. All calls, whether local or remote, are wrapped in the `@retry_on_429` decorator, ensuring automated graceful degradation instead of failed requests.
6. Results are aggregated by the Orchestrator into the final response format and persisted to disk (`chat_history.json`, `records.json`).

This consistent trigger discipline is what lets Mesh AI mix eight very different capabilities (research, vision, documents, audio, slides, email, expenses, and memory) without the system becoming fragile or unpredictable.

---

5. Tech Stack

- **Backend:** FastAPI (central gateway), Python
- **Agent Orchestration:** Gemini 2.5 Flash / Gemini 2.5 Flash-Lite
- **Protocol Layer:** Model Context Protocol (MCP) via FastMCP servers, SSE (`ClientSession`)
- **Frontend:** Next.js dashboard
- **Presentation Generation:** `python-pptx`
- **Charting:** Matplotlib with Pillow (PIL) fallback
- **Search Resilience:** DuckDuckGo API, then Wikipedia API, then offline mock compiler
- **Persistence:** JSON-based state files (`chat_history.json`, `records.json`)

---

6. Differentiation

Most hackathon agent projects wire a single LLM to a handful of tools and hope the demo doesn't hit a rate limit. Mesh AI is built differently:

- **Genuine multi-agent separation of concerns:** A Research Agent and a Compliance Writer Agent are distinct, chained agents, not one model wearing different prompts.
- **Isolation via MCP:** Search and mail run on independent FastMCP servers, so a failure or slowdown in one doesn't take down the core orchestrator.
- **Engineered for failure, not just for the happy path:** Three-layer search fallback, binary-conflict-aware charting, and exponential backoff on every external call mean

the system was designed to survive exactly the conditions a live demo (or a real office) throws at it.
- **Multimodal breadth in one mesh:** Text, images, PDFs, audio, and generated slides all flow through the same orchestrator and the same memory layer, rather than being separate bolt-on features.

---

7. Roadmap

- Expand the Mock Entity Database into a live CRM/directory integration for real contact and team routing.
- Add a scheduling agent that can propose and stage calendar invites alongside drafted emails.
- Extend the Compliance Writer Agent to support configurable output templates per industry (finance, legal, sales).
- Introduce a lightweight permissions layer so the Auto-Mail Engine can safely operate with real send access, not just staging.
- Add multi-user support to the Memory History Engine for shared team context.

---

8. Conclusion

Mesh AI demonstrates that a business-facing agent system doesn't have to choose between capability and reliability. By combining a hub-and-spoke orchestration model, MCP-based isolation, genuine multi-agent pipelines, and a resilience layer built into every external call, Mesh AI turns a set of eight distinct business tasks (intelligence gathering, document and image understanding, meeting summarization, presentation generation, email drafting, expense tracking, and persistent memory) into one coherent, dependable assistant. It's built not just to work in a demo, but to keep working when the APIs don't cooperate.